Simon Edmunds
Intro to Computer Vision - CS 5404
10/8/2025

Homework 2: Features, Scaling and Warping

2.1.1 - Computing Harris Corners

Shown below are plots demonstrating the process of computing the Harris corners for two different images:

Image 1:

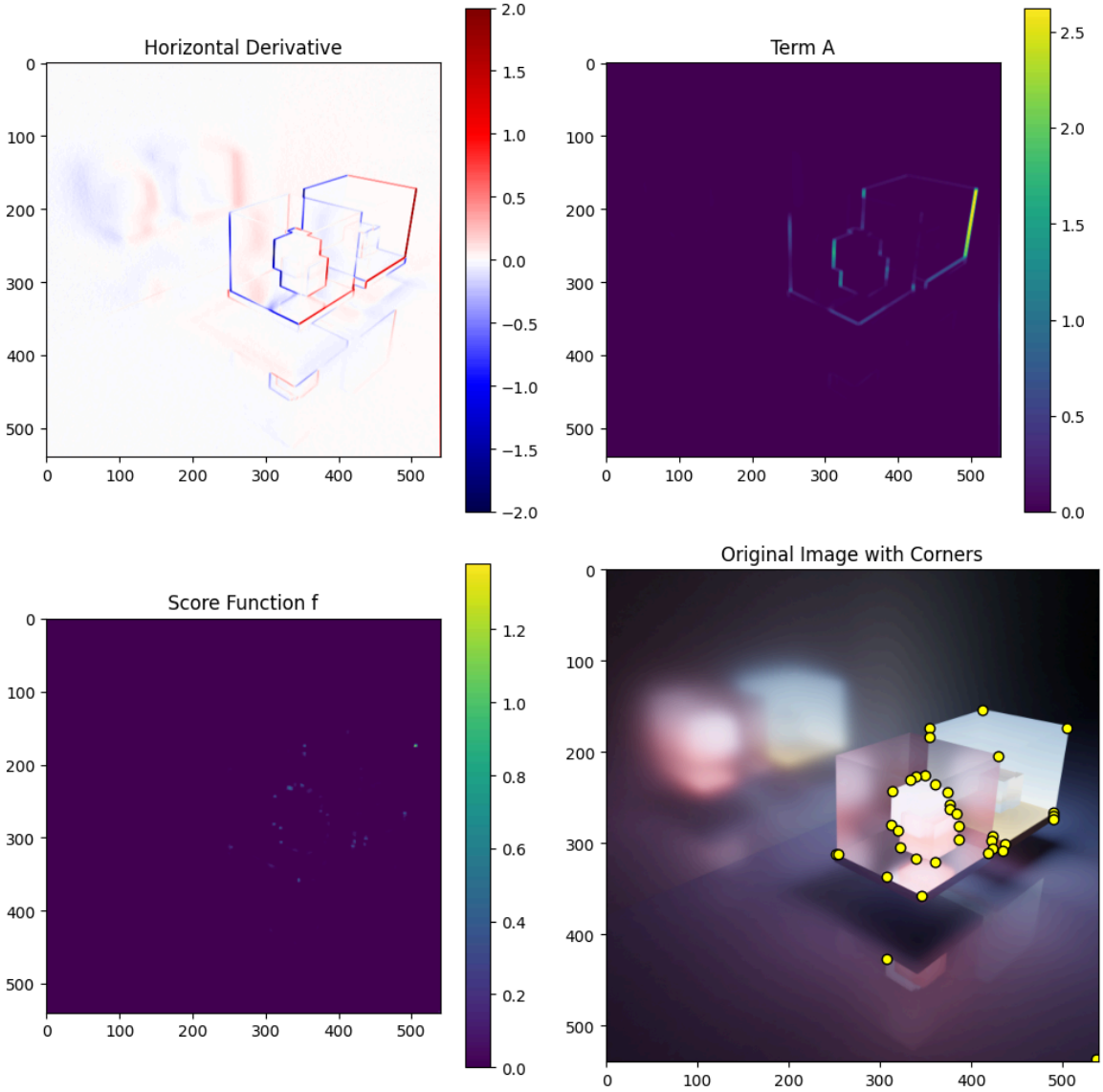
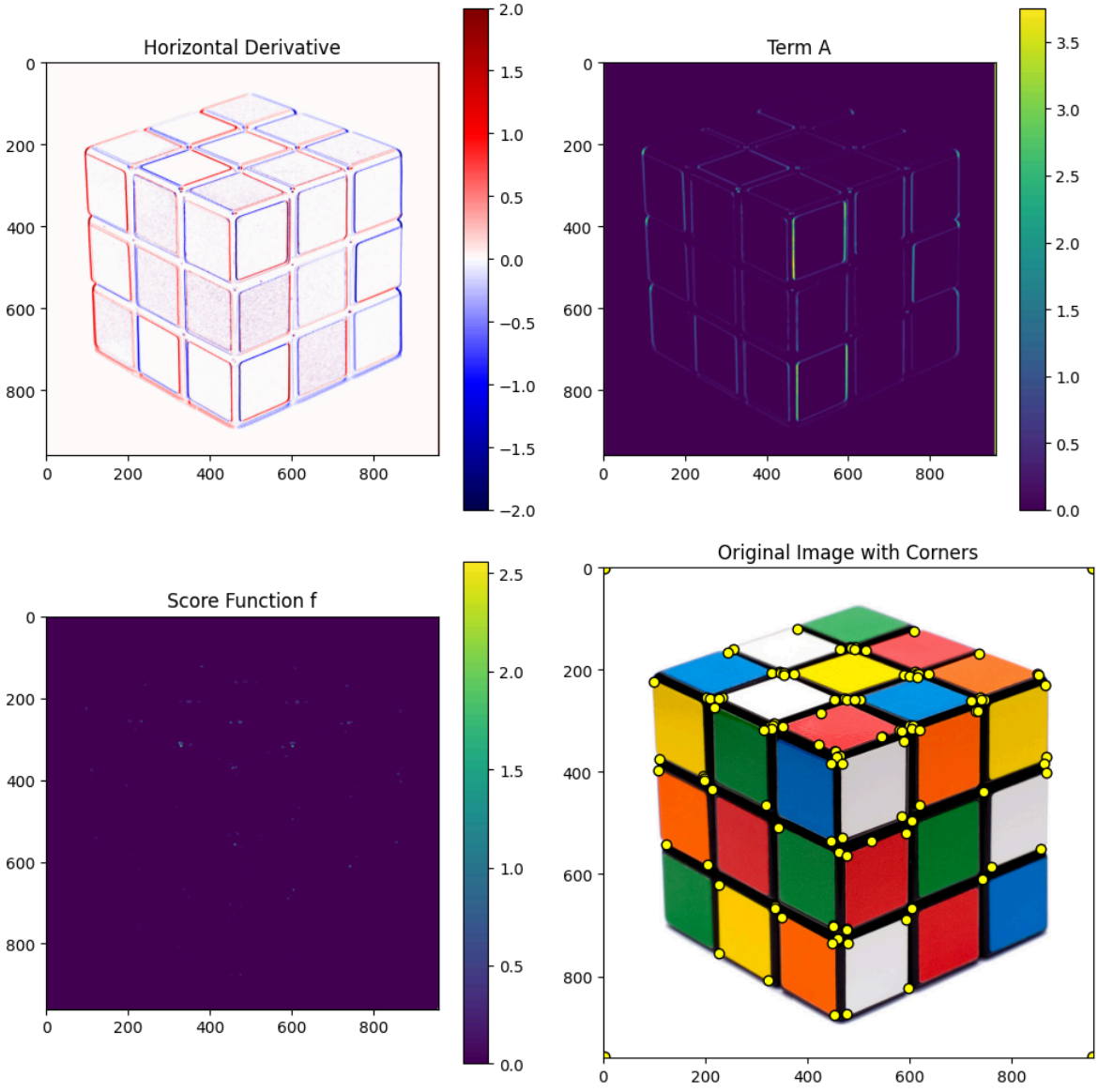
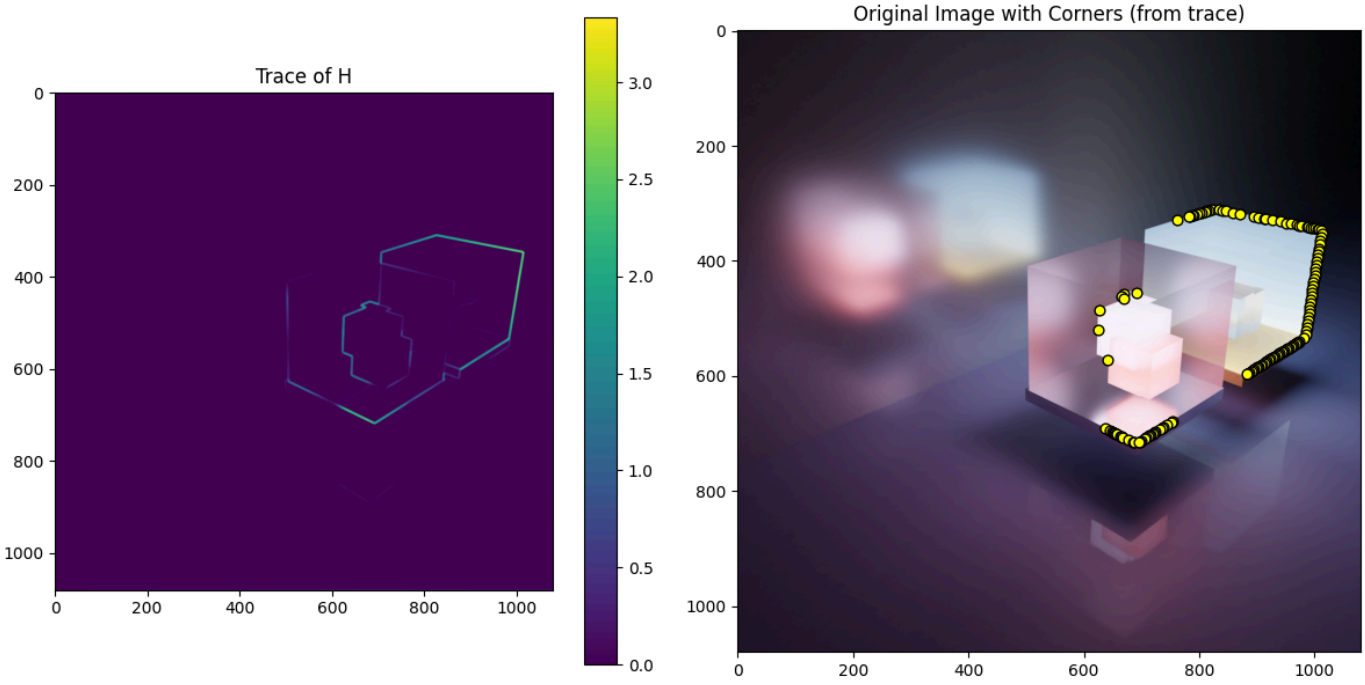


Image 2:



Question: Are there any features in the Light Cubes image that you are surprised are *not* present? Highlight or discuss in words one or two regions of one of your images where features were detected that you did not expect, or one or two regions you thought features might exist but do not.

In the Light Cubes image, I was surprised that only one corner was detected in the floor reflection, despite the clearly visible sharp angles. In image 2 (the Rubik's Cube), no corners were detected at the cube's bottom-left, bottom-right, and top-center corners, as well as some sticker edges, which was surprising since these appear as distinct corners to the human eye, but scored weakly in the detection function.

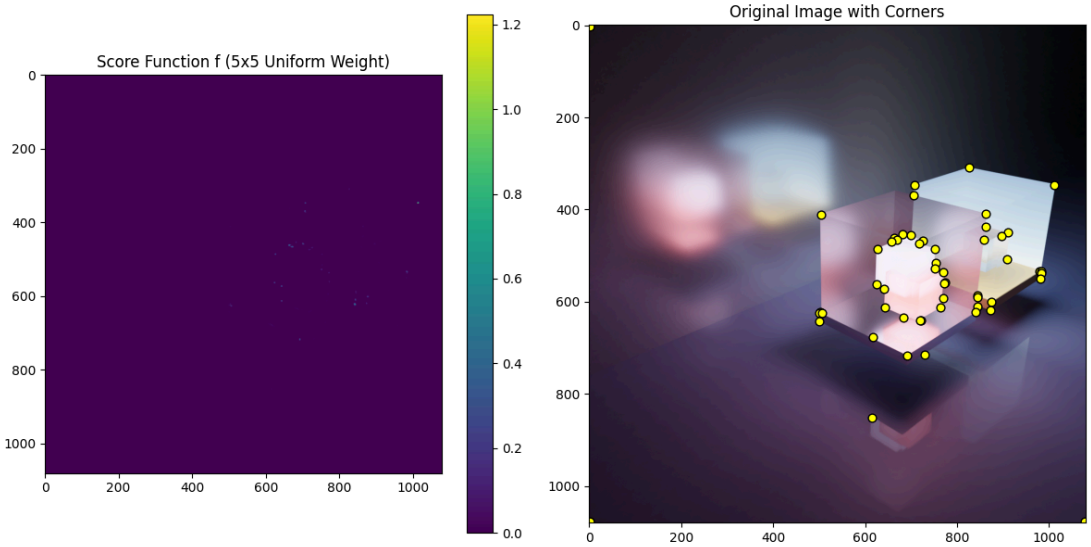


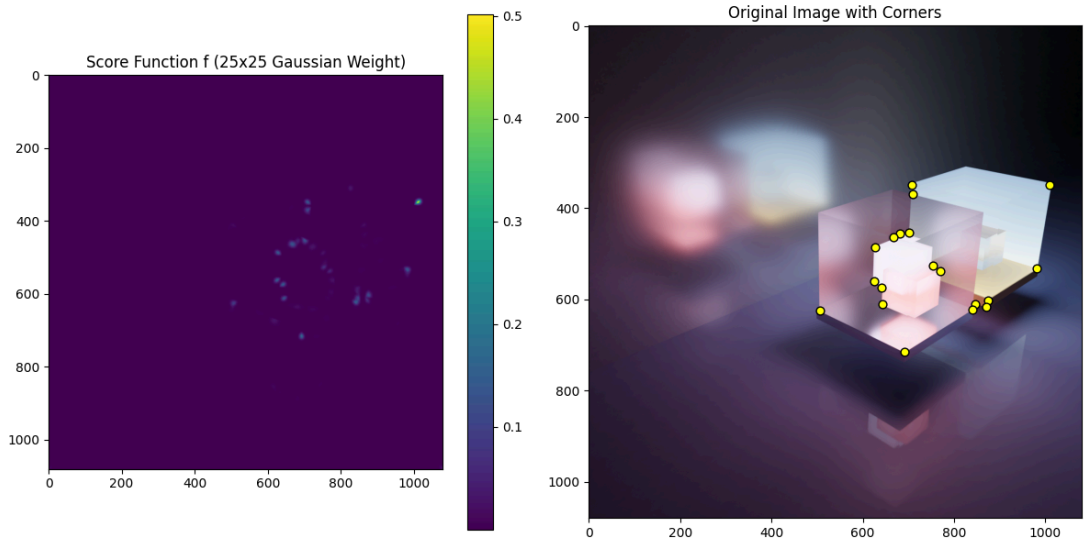
Question: What does this modified scoring function (using the trace of H) detect? If we want to detect corners, why might we not want to use this scoring function?

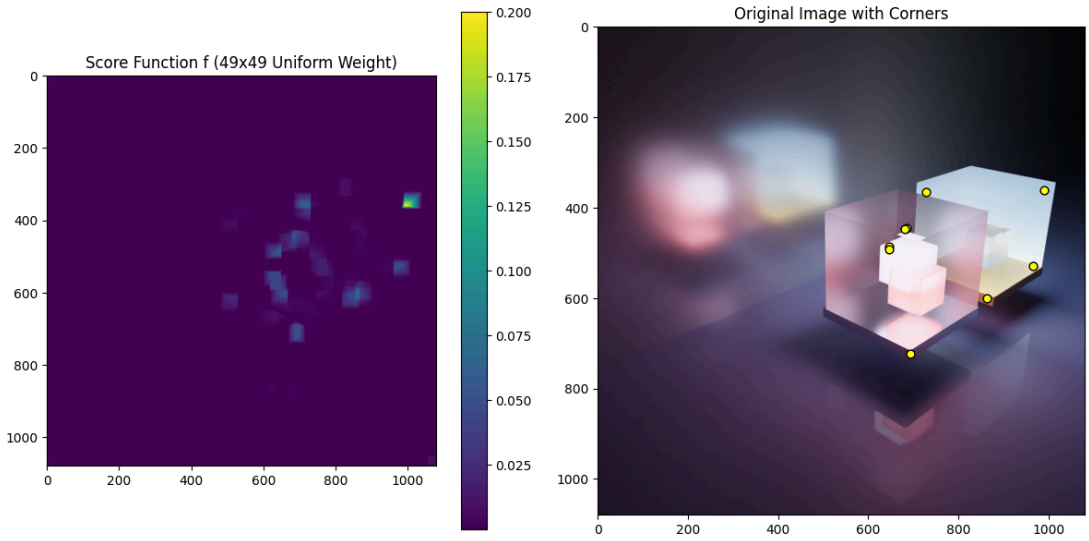
After modifying the scoring function, it now detects edges rather than corners. If the goal is to detect corners specifically, this scoring function is not ideal because it responds to all changes in gradient, not just the specific patterns at corners.

2.1.2 - Varying the Weight Matrix

Below are the scoring functions and corners detecting using three differently sized weight matrices:







Question: (3-5 sentences) Discuss the differences between these three weight functions. In particular, how do the features change for the third kernel, which is significantly 'wider' than the others?

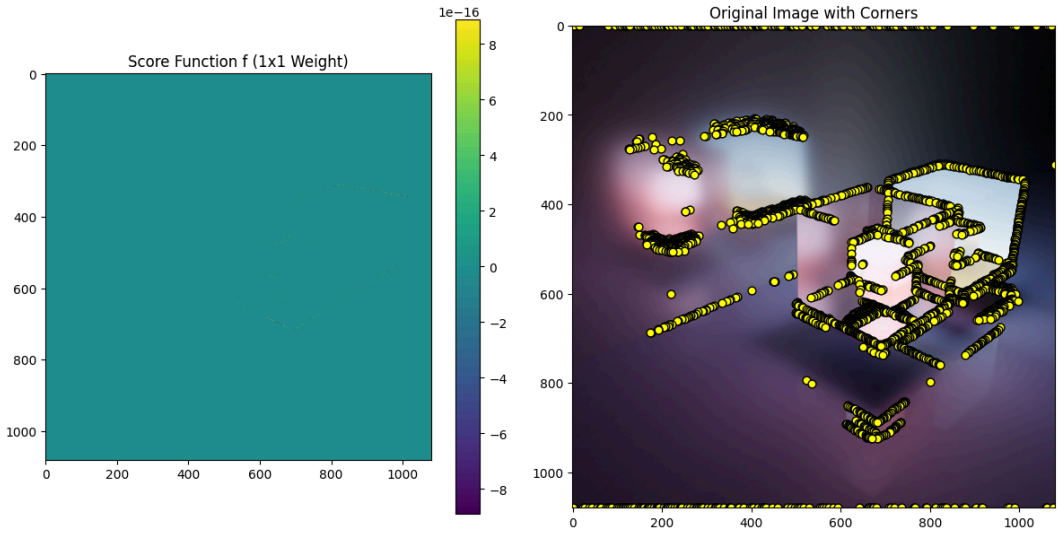
The small 5×5 uniform kernel (the first image) detects many small corners, but sometimes finds many corners in one spot, or treats what a human would consider an edge as a corner. The 25×25 Gaussian kernel smooths over a larger area, so it finds fewer of the small corners, focusing on large, prominent corners. The 49×49 uniform filter (the third image) averages over a very large region, which removes even more detail. This causes it to detect only the most prominent corners in the image, and the points are shifted slightly away from the actual corner position, due to the huge averaging window.

Question: What happens to the score function if we were to use a 1x1 weight matrix?

$$w = [1]$$

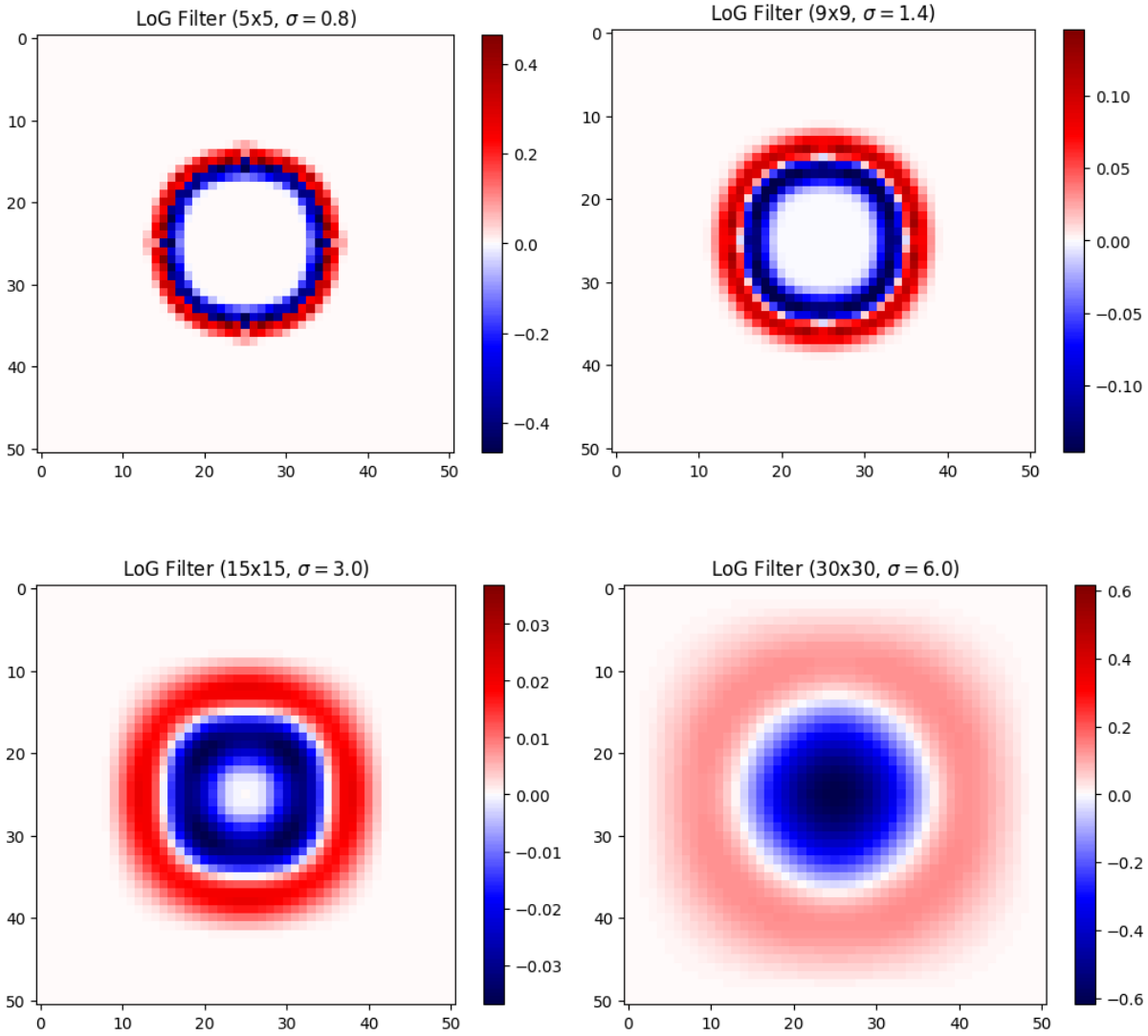
Does this make sense? (Hint: think back to how we have defined and measured "cornerness"? Does it make sense to determine cornerness with a 1x1 window?)

If you were to use a 1x1 weight matrix, there would be no smoothing at all (since there isn't really a "window"). This means that the score function would simply respond to any gradient in the image, effectively becoming an edge detector, as shown below:

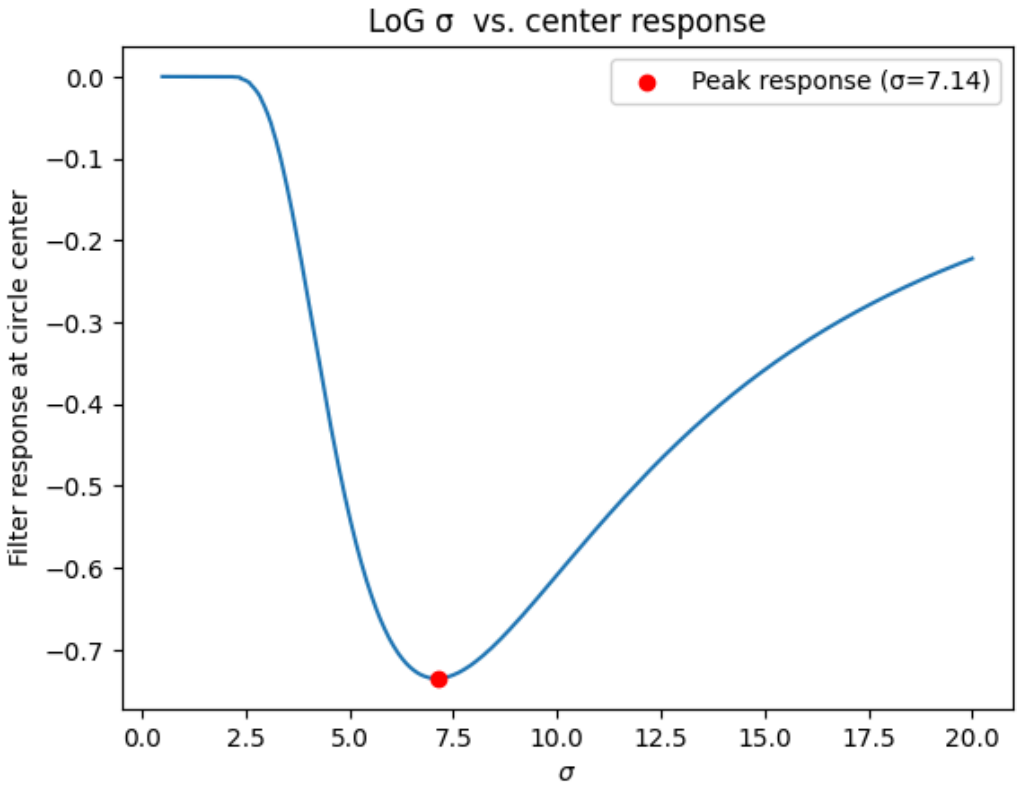


2.2.1 - Scale-Normalized Filter Response

The below images show the result of applying normalized Laplacian of Gaussian (LoG) filters at different scales to a sample circle image.



Below is a plot of the values of σ versus the LoG filter response at the center of the circle image. The plot also highlights the value of σ that produces the maximum response.

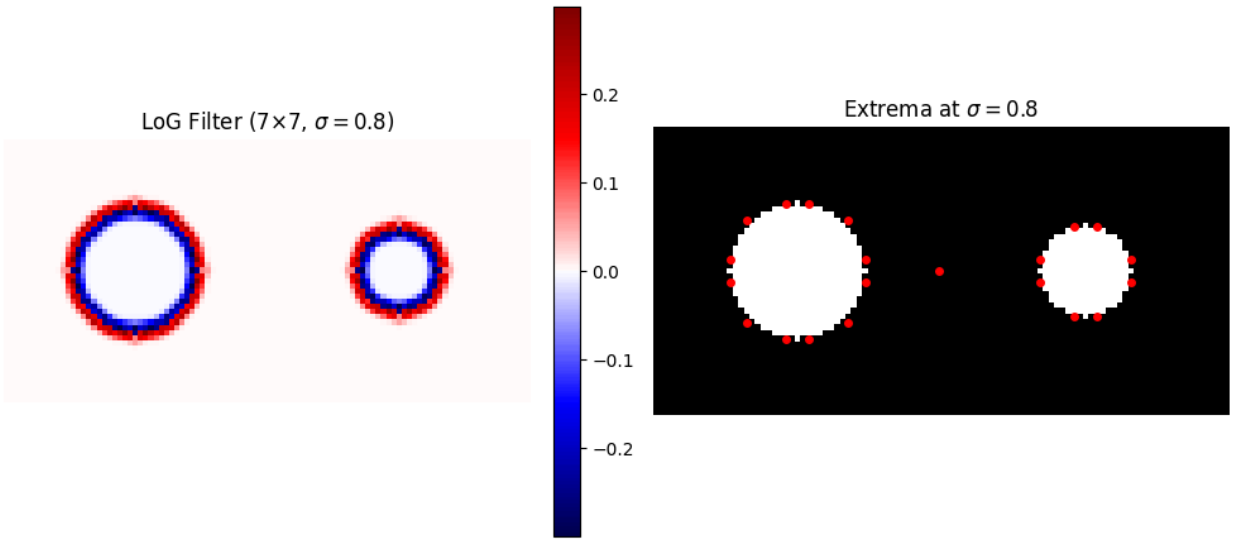


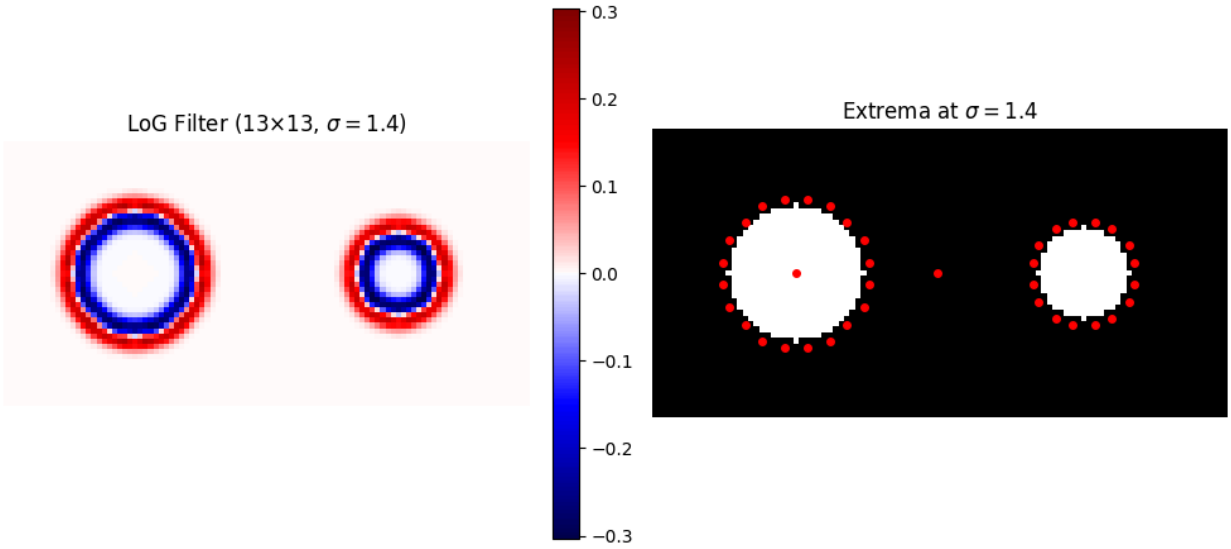
Question: What is the relationship between the peak σ and the circle's radius?

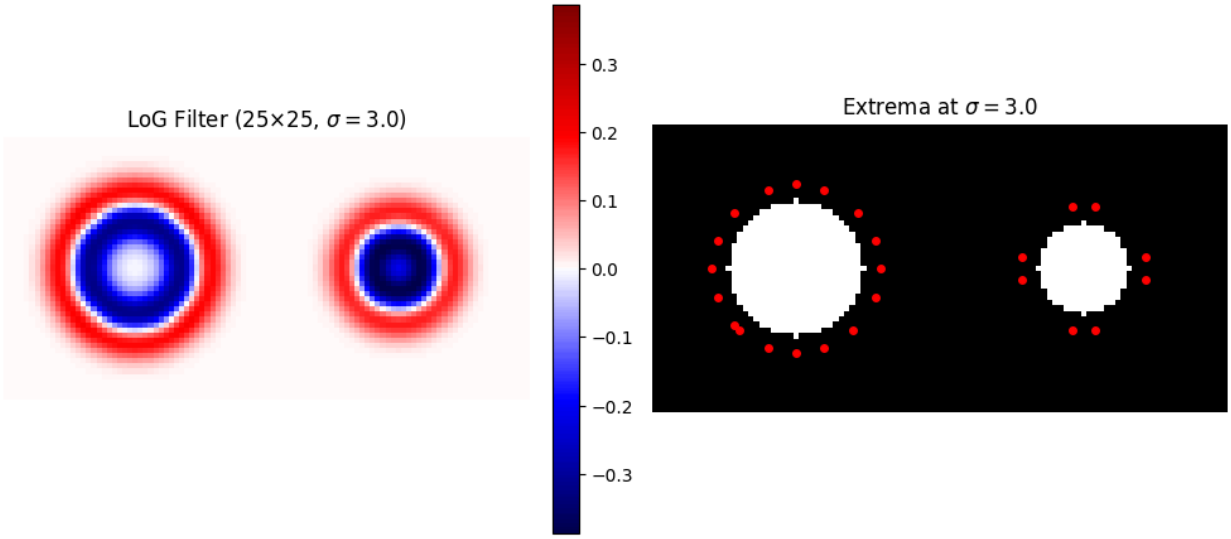
$$\sigma_{\text{peak}} \approx \frac{R}{\sqrt{2}} \approx 7.07 \quad \text{(theory)}, \quad \sigma_{\text{peak}} \approx 7.14 \quad \text{(measured)}$$

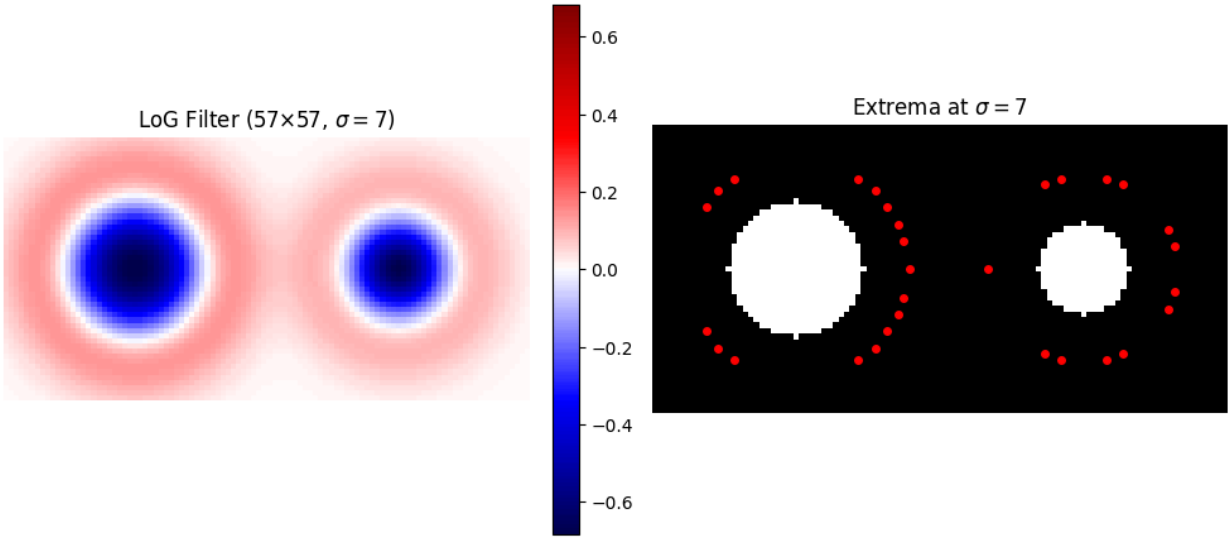
2.2.2 - Annotating an Image with Multi-Scale Detections

Below are plots of the scaled Laplacian of Gaussian filter for different values of sigma, as well as plots of the detected features from the extrema of the filter:

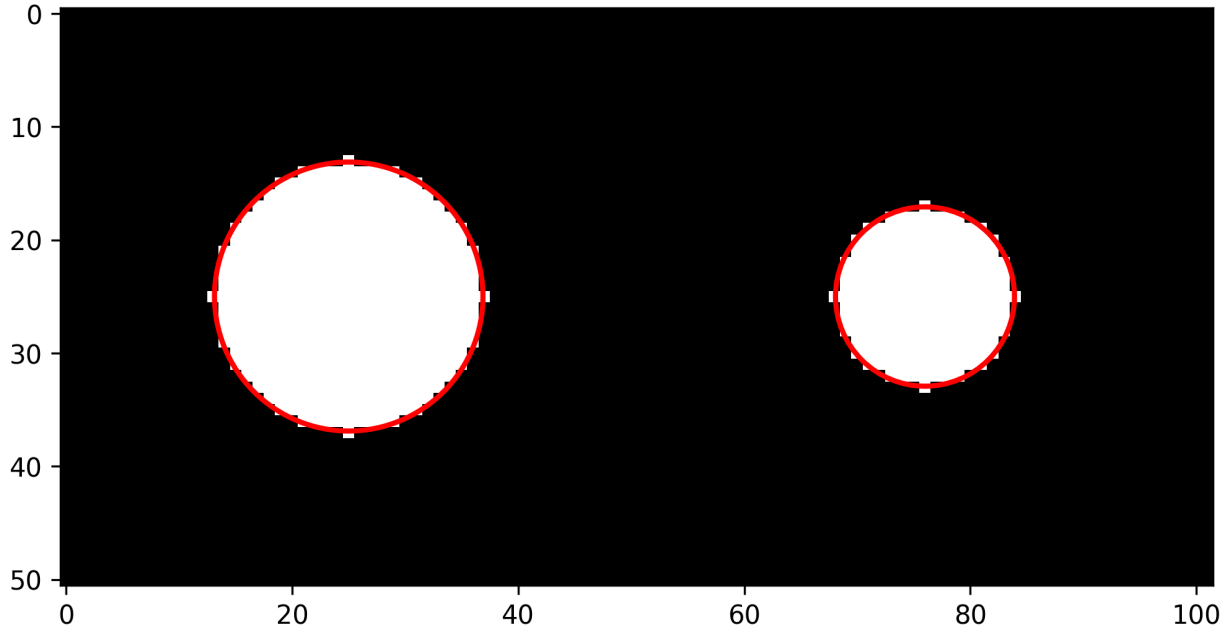


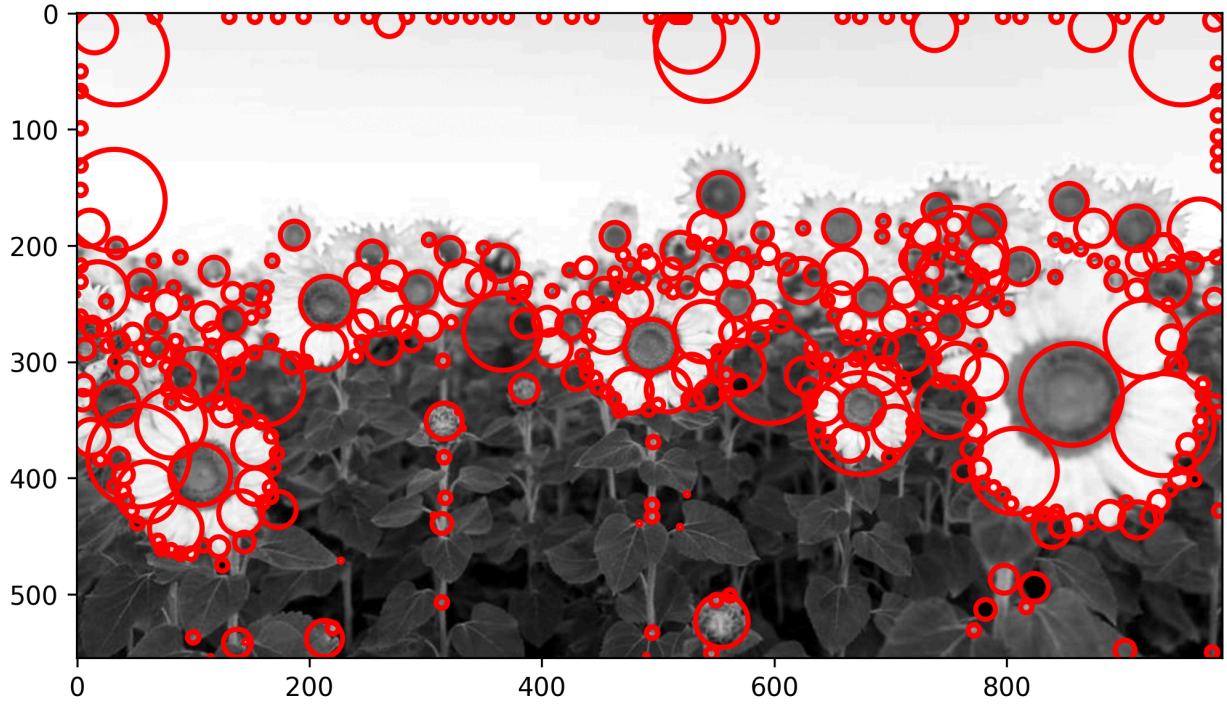


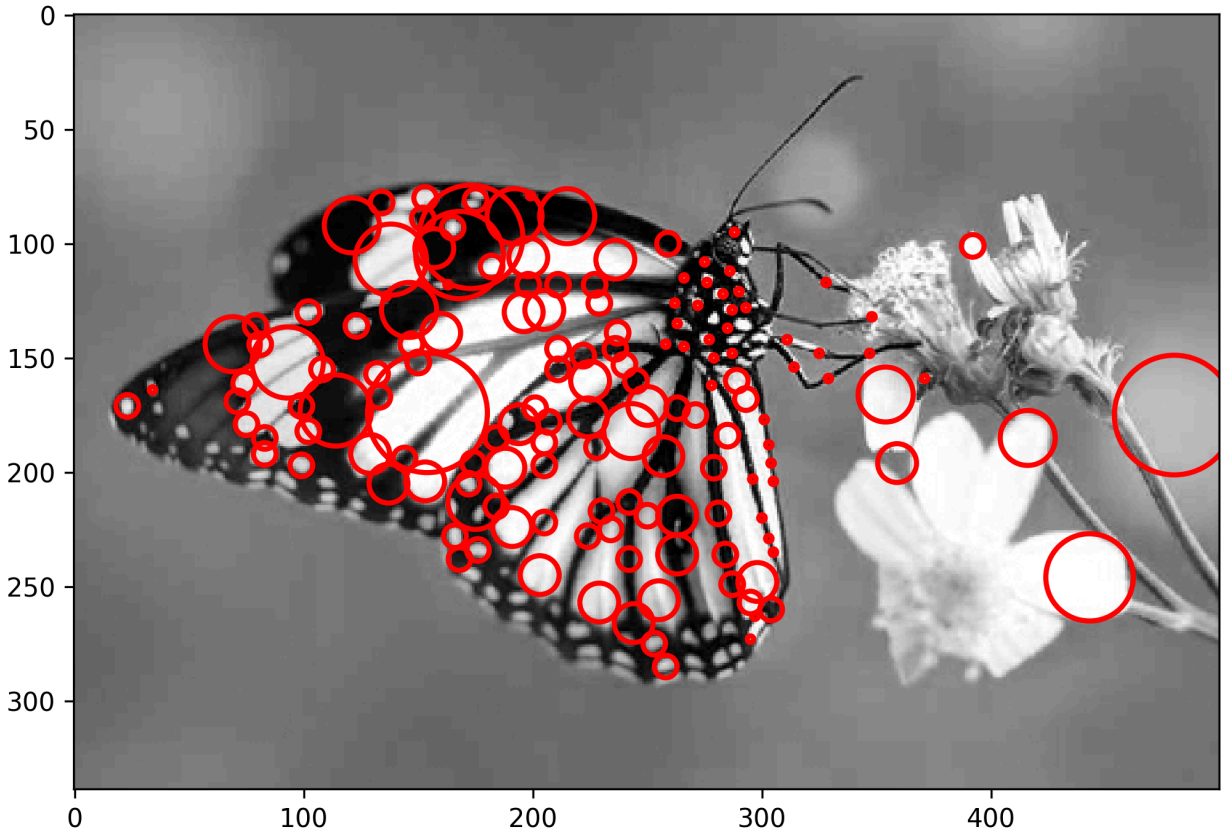

LoG Filter (25×25, $\sigma = 3.0$)
0.3
0.2
0.1
0.0
−0.1
−0.2
−0.3
Extrema at $\sigma = 3.0$


LoG Filter (57×57, $\sigma = 7$)
0.6
0.4
0.2
0.0
−0.2
−0.4
−0.6
Extrema at $\sigma = 7$

Below are plots of features detected using multi-scale blob detection for three different example images:





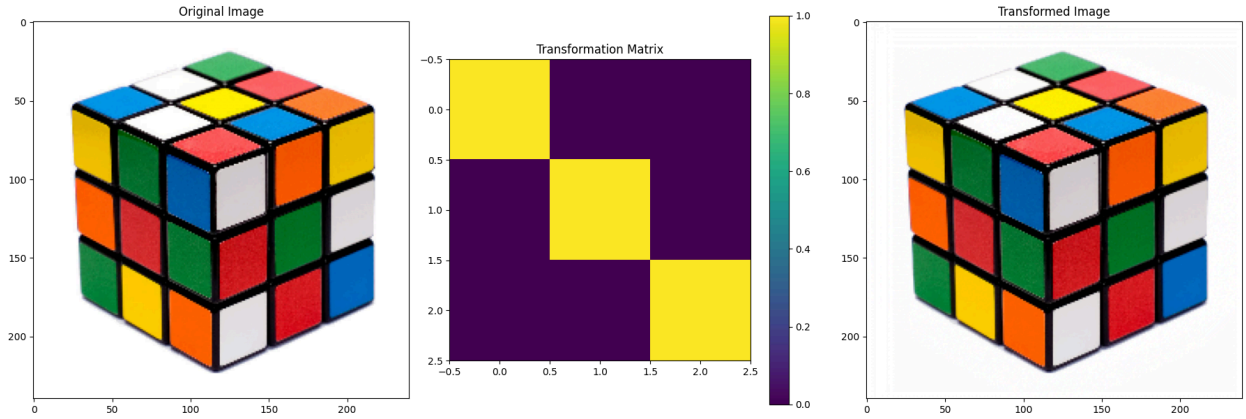

0
50
100
150
200
250
300
0
100
200
300
400

2.3 - Image Warping

1. Identity Filter

This filter behaves exactly as expected: It simply leaves the image unchanged, giving the exact same output as input.

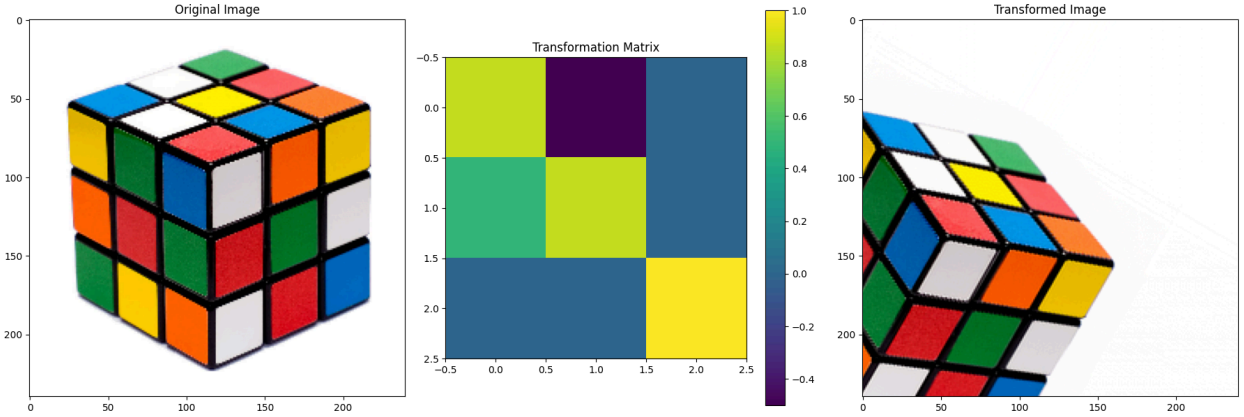
$$T_{\text{identity}} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$



2. 30° Rotation

The below matrix rotates the image by 30°. One important thing to note is that this rotation is applied around the origin of the image, which is the top left corner, *not* the center. This causes the image to be rotated slightly "off-screen".

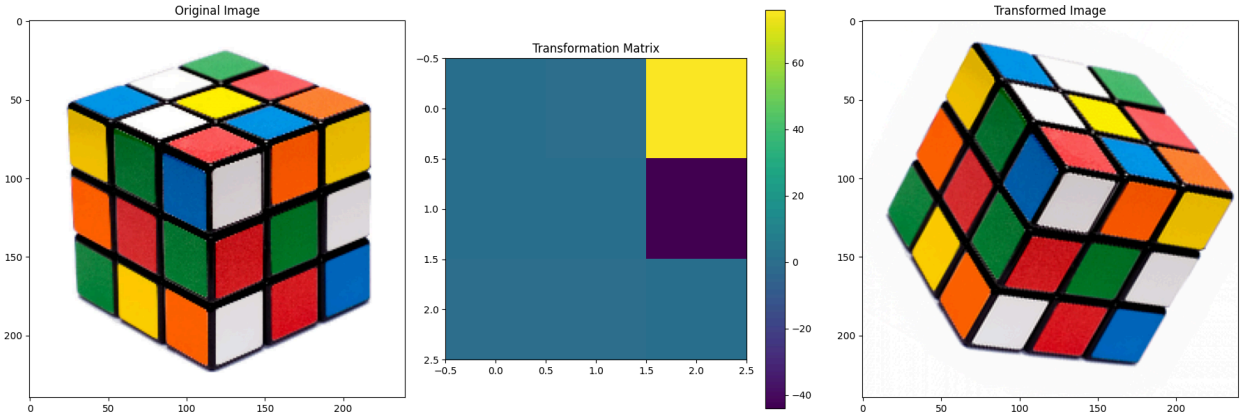
$$T_{\text{rotation}} = \begin{bmatrix} \cos(30^\circ) & -\sin(30^\circ) & 0 \\ \sin(30^\circ) & \cos(30^\circ) & 0 \\ 0 & 0 & 1 \end{bmatrix}$$



3. **30° Rotation + Translation to Image Center**

   This matrix behaves almost exactly like the previous one, except I added an offset in both the x and y directions, in order to center the image on the visible plot. This offset is hard-coded, but a valid offset could be found for any image using trigonometry.

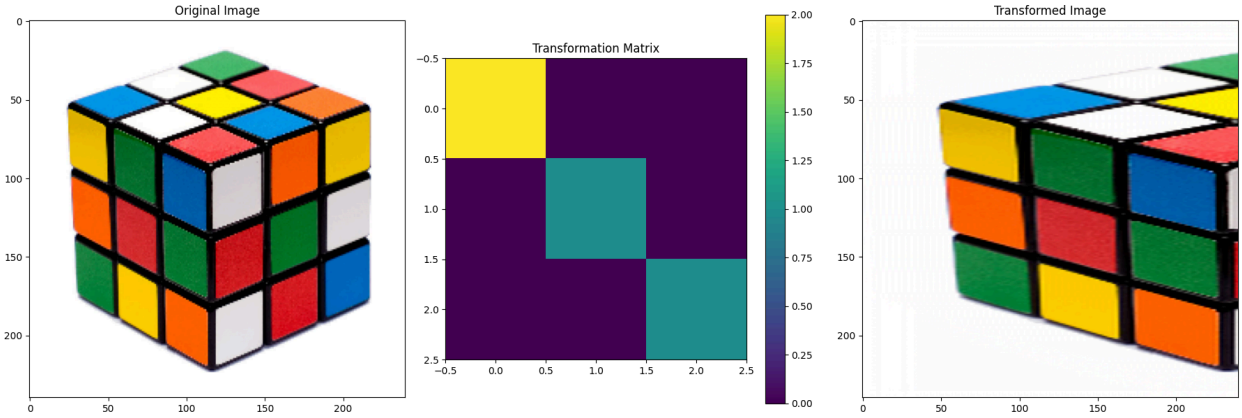
$$T_{\text{rot+trans}} = \begin{bmatrix} \cos(30^\circ) & -\sin(30^\circ) & 76 \\ \sin(30^\circ) & \cos(30^\circ) & -44 \\ 0 & 0 & 1 \end{bmatrix}$$



4. **Scale X-Axis by 2**

   This matrix performs a scaling operation on the image, visually "stretching" it horizontally. Because the transform code we were provided keeps the output image the same size, a portion of the stretched image is cut off.

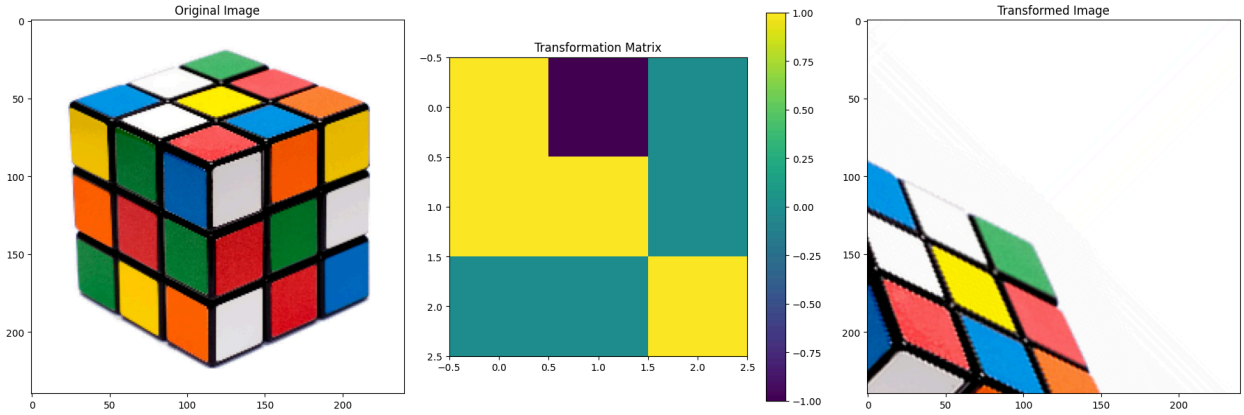
$$T_{\text{scale}} = \begin{bmatrix} 2 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$



5. **“Mystery” Kernel**

   This provided matrix does more than one transformation. At first I thought it looked similar to a shear matrix, but as I show below, it isn’t doing a shear transformation at all; it’s actually scaling the image in both x and y direction by a factor of $\sqrt{2}$, and then applying a 45° rotation.

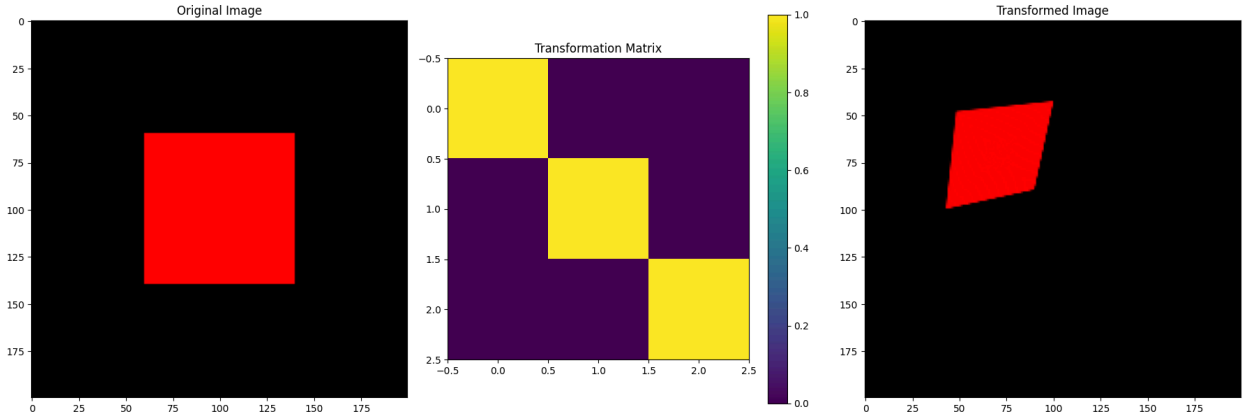
$$T_{\text{special}} = \begin{bmatrix} 1 & -1 & 0 \\ 1 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} \sqrt{2} & 0 & 0 \\ 0 & \sqrt{2} & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} & 0 \\ \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} \sqrt{2} & 0 & 0 \\ 0 & \sqrt{2} & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos 45^\circ & -\sin 45^\circ & 0 \\ \sin 45^\circ & \cos 45^\circ & 0 \\ 0 & 0 & 1 \end{bmatrix}$$



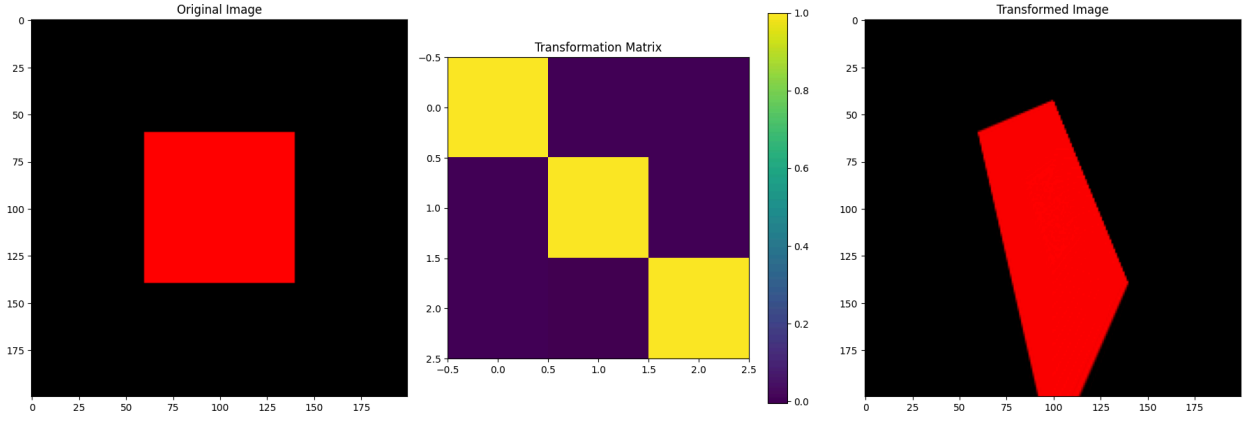
Homography Transforms

Below are two examples of homographic transformations applied to an example image.

$$T_{H1} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0.002 & 0.002 & 1 \end{bmatrix}$$



$$T_{H2} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0.005 & -0.005 & 1 \end{bmatrix}$$

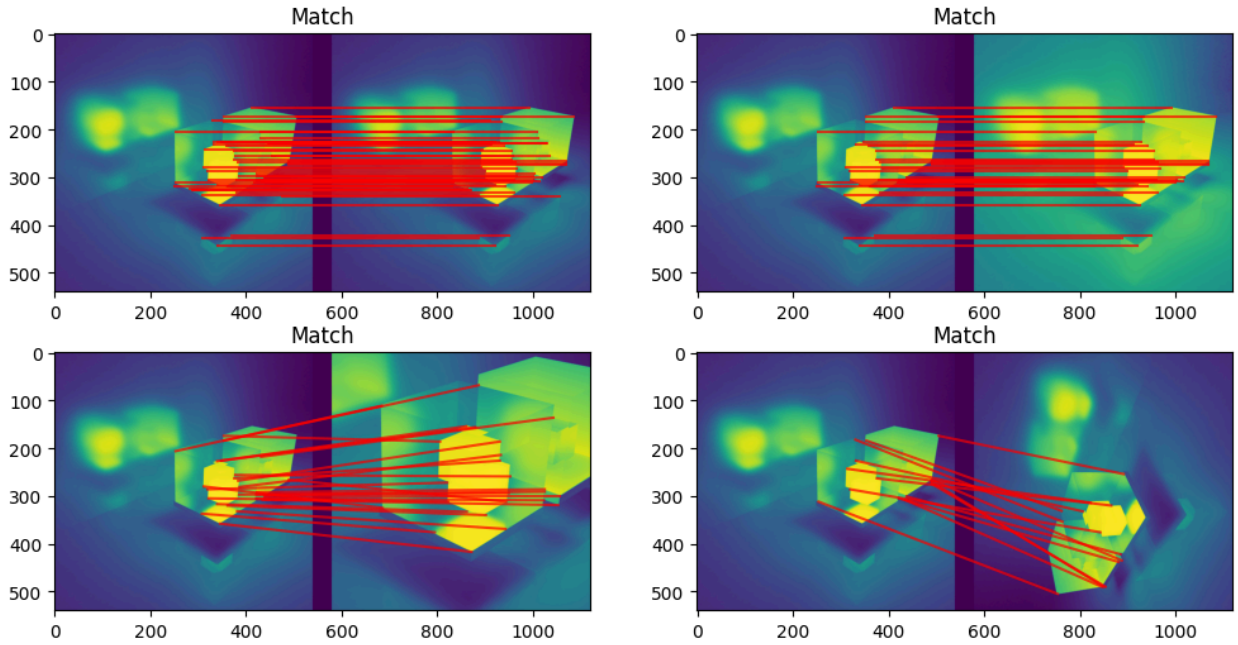


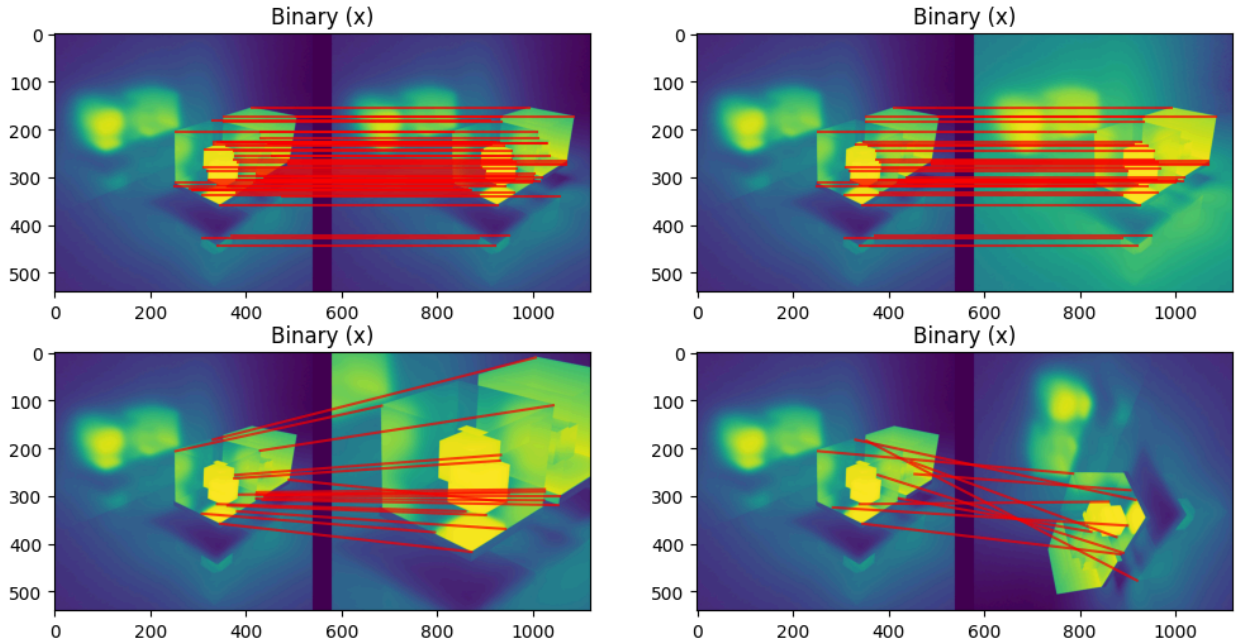
Question: How do the two "bottom row" parameters control how the image is warped?

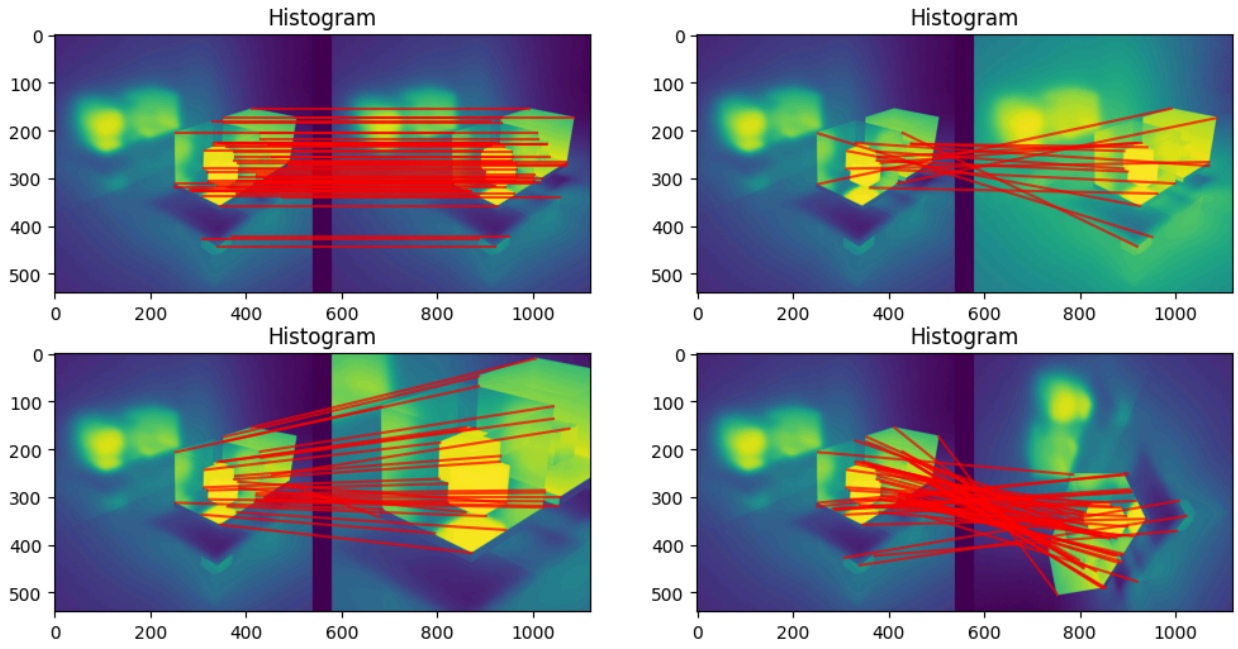
The two bottom-row parameters affect how the image is scaled across the coordinates, which adds a “perspective warp”, making it look like you are viewing the image from another angle. The first (left-most) element in the bottom row affects scaling in the x-direction, and the second affects scaling in the y-direction.

2.4 - Some Simple Feature Descriptors

Below are three figures demonstrating different feature matching methods using the Harris corner detection from earlier in this assignment:







Question A: Which feature descriptor performs poorly on the image `img_contrast`? Explain why this descriptor performs worse than the others.

The histogram-based feature descriptor performs very poorly on `img_contrast`, with almost no accurate matches. This makes sense because a histogram descriptor stores the distribution of intensity values across the window. However, when you apply a contrast filter, all of those intensities shift dramatically, causing the histogram values to no longer be useful.

Question B: Which feature descriptor performs best on the image `img_transpose`? Explain why this descriptor performs better than the others.

The histogram-based feature descriptor performs the best on `img_transpose`. This is because the histogram descriptor only considers the intensity distribution of a feature, not how it is oriented/positioned. Because intensity distributions do not change during a transpose, these features are preserved, and the histogram remains accurate.